System Verification and Validation Plan for Software Engineering

Team 8, AlgoCatan
Jake Read
Rebecca Di Filippo
Matthew Cheung
Sunny Yao

April 6, 2026

Revision History

Date	Version	Notes
10/19/2025	Draft	Initial VnV plan without Unit Testing
04/05/2026	Final	Feedback from TA for final documentation

Contents

List of Tables

1 Symbols, Abbreviations, and Acronyms

The following glossary headers are hyperlinked to their online definitions for further reading.

- Catan – A strategy board game called *Settlers of Catan* where players collect resources, build roads/settlements, and trade to earn points.[1]
- AI – Field of computer science and engineering that focuses on creating systems capable of performing tasks that usually require human intelligence.[2]
- RL – A type of machine learning where an agent, known as Reinforcement Learning, learns by interacting with an environment and receiving rewards or penalties for its actions.[3]
- Digital Twin – A digital system that mirrors a physical one. In this project, it refers to a digital representation of the physical *Catan* board, updated in real time.[4]
- CV – An area of AI that trains computers to interpret and understand visual information, such as the physical *Catan* board.[5]
- LLM – A machine learning model trained on large amounts of text data to generate and understand language.[6]
- Game State – The current configuration of the game, including player resources, board layout, and dice rolls.[7]
- FR – A requirement that defines a function of the system; it specifies what the system must do.[8]
- NFR – A requirement that specifies a quality attribute of the system; it defines how the system must be.[9]

This document presents the Verification and Validation (VnV) plan for AlgoCatan. The plan has been updated and refined since the MIS (detailed design document) was completed.

2 General Information

2.1 Summary

The software being tested is AlgoCatan, an AI system for training, serving, and interacting with *Catan* bots through a web-based interface. The implemented system includes a reinforcement learning training pipeline, an AI model, an Elo league system, a curriculum learning manager, a backend API server, a game state manager, a game state database, a user interface, and an explanation pipeline that generates natural-language descriptions of bot decisions.

The general purpose of AlgoCatan is to provide users with skill-matched bot opponents, replay and analysis tools, and natural-language explanations of strategic decisions. Computer vision for reading a physical board is deferred and not part of the implemented system being validated in this document.

2.2 Objectives

The objective of this V&V plan is to build confidence in the correctness, reliability, and functional performance of the AlgoCatan system by verifying that all implemented system components satisfy the requirements defined in the Software Requirements Specification (SRS).

Verification will focus on ensuring that the reinforcement learning agent generates valid moves and that data exchange between system components is accurate and consistent.

The V&V plan is primarily derived from the Software Requirements Specification (SRS), which defines all functional and non-functional requirements used to construct system-level test cases. In addition, the Hazard Analysis document is used to prioritize verification effort toward safety-critical and failure-prone components, particularly in the game state synchronization logic. System architecture and design documentation further support

test design by identifying component boundaries, data flow points, and integration interfaces that must be validated during system integration testing.

Out of scope are extensive performance benchmarking beyond defined acceptance thresholds, as these exceed the project's time and resource constraints. The Computer Vision module is also out of scope, as it was not implemented in the final system; therefore, no verification activities are performed for CV functionality. External dependencies such as Catanatron and the OpenAI Gym environment are not verified internally; instead, verification focuses on correct integration and usage within the AlgoCatan system.

2.3 Challenge Level and Extras

The challenge level for this project is Advanced, as outlined in the problem statement and confirmed with the course instructor. This project reflects the complexity of developing a reinforcement learning agent capable of mastering *Settlers of Catan*, while also integrating a training pipeline, curriculum learning, Elo-based opponent selection, a web interface, backend APIs, replay support, and an LLM-based explanation feature.

The approved extras for this project include:

- User Instructional Video: A walkthrough video that explains how to set up, run, and interact with the software, helping users understand its functionality and workflow.
- Usability Testing Report: A comprehensive report summarizing the results of usability testing sessions, including user feedback, identified issues, and recommendations for improving the user experience.

2.4 Relevant Documentation

The following documents provide essential background for the VnV plan:

1. **Software Requirements Specification (DiFilippo et al. (2025b)):** This document outlines the functional and non-functional requirements for AlgoCatan, serving as the primary reference for the verification and validation plan.

2. **Hazard Analysis (DiFilippo et al. (2025a)):** This document identifies potential system-level hazards, their causes, and their effects. The functional and non-functional safety requirements derived in the Hazard Analysis are the basis for many of the critical system tests in this VnV plan.

3 Plan

This section outlines the strategy for Verification and Validation (VnV) across all phases of the AlgoCatan project lifecycle. The plan details the roles and responsibilities of the VnV team, and specifies structured approaches for verifying the Software Requirements Specification (SRS), Design, and Implementation, as well as the strategy for Software Validation. This systematic approach is designed to increase confidence in the correctness, functional performance, and reliability of the final system. (insert visual roadmap)

3.1 Verification and Validation Team

The VnV efforts will be a collaborative team effort, with specific responsibilities assigned based on existing project roles to ensure systematic coverage and accountability.

Table 1: Verification and Validation Team Roles

Team Member	**Project Role**	**VnV Responsibilities**
Jake Read	Team Leader	Coordinating all VnV activities, performing final review of all test cases, and leading the code walkthrough for the supervisor.
Rebecca Di Filippo	Notetaker	Documenting VnV meeting minutes, maintaining traceability tables, and preparing the Usability Survey for validation.
Sunny Yao	IT/DevOps	Implementing and maintaining CI/CD pipelines, monitoring automated test coverage metrics, and troubleshooting technical VnV tool issues.
Matthew Cheung	Researcher	Researching and applying non-dynamic testing techniques (e.g., formal code inspection checklists) and collecting external data for software validation.
Dr. Istvan David	Project Supervisor	Providing technical oversight, expert consultation on *RL* model correctness, and participating in the Revision 0 demonstration for requirements validation.

3.2 SRS Verification

The SRS will be verified against the project's objectives to ensure all requirements are unambiguous, correct, feasible, and complete (Quality System Tests).

- **Structured Review with Supervisor:** A dedicated meeting will be scheduled with Dr. Istvan David, the Project Supervisor. The team will present the consolidated functional, non-functional, and safety requirements, specifically focusing on the formalizations (e.g., Z-Notation) and the key constraints (e.g., ≤ 1 second response time for advice). Specific questions will be prepared beforehand to validate the technical accuracy of the *RL* requirements (e.g., reward function structure).
- **Peer Review and Inspection:** Classmates and primary reviewers will be engaged to perform an ad-hoc review of the SRS, targeting

clarity and consistency. We will aim for a minimum of 5 reviews, ranging across the whole document.

- **Issue Tracker for Traceability:** All feedback and identified issues will be logged on the GitHub Issues board, assigned to a team member, and traced back to the specific requirement for resolution.
- **Checklist-Based Inspection:** The team will employ a custom checklist (developed by the Researcher) focusing on the characteristics of a high-quality requirement (unambiguous, verifiable, complete) to systematically inspect the SRS.

3.3 Design Verification

Design verification will occur after the Design Document is completed (Milestone: Nov. 10) to ensure the architectural design meets all requirements and is ready for implementation.

- **Interface Inspection:** The Researcher will lead an inspection of the implemented component interfaces, such as the request/response contracts between the User Interface and the Backend API Server, and the explanation request/response flow between the Backend API Server and the Explanation Pipeline. This ensures modularity and that the design supports FR and NFR related to communication and data exchange.
- **Structured Peer Review:** The design will be reviewed by all team members to check for feasibility, proper component decomposition, and adherence to design principles (e.g., modularity).
- **External Review:** Classmates will be asked to review the design, focusing on potential bottlenecks or single points of failure, particularly regarding real-time operation and *AI* performance constraints.

3.4 Verification and Validation Plan Verification

The VnV Plan itself is a critical project artifact that must be verified for feasibility, clarity, and completeness.

- **Team Review:** The plan will be reviewed by all team members, ensuring that the defined VnV tasks are feasible within the academic timeframe and that the level of detail is sufficient for execution.

- **Feasibility Assessment:** The Team Leader will specifically check that the execution of the full VnV plan is realistic given the eight-month project timeline and the team's current skill set.
- **External Feedback:** The plan's clarity and completeness will be verified by the course TA and classmates during informal review sessions.

3.5 Implementation Verification

Implementation verification is focused on ensuring the source code correctly implements the design and fulfills the requirements. This will use a combination of dynamic (automated tests) and non-dynamic (static analysis and inspection) techniques.

- **Unit Testing and System Tests:** All functional and non-functional requirements will be verified through the comprehensive dynamic tests described in the System Tests and Unit Test Description sections of this document.
- **Static Analysis and Linters:** The team will enforce coding standards (PEP 8 for Python, Google Style Guide for JavaScript/React) using automated linters and static analyzers (SonarQube).
- **Peer Code Inspection:** All code changes will require a review and approval via GitHub Pull Requests. Team members will specifically check for adherence to coding standards, algorithm correctness, and proper error handling.

3.6 Automated Testing and Verification Tools

Automated VnV tools are central to maintaining code quality and continuous integration.

- **Unit Testing Frameworks:** Python's built-in `unittest` or `pytest` were planned for backend unit testing, and Jest was planned for the frontend React components. However, during implementation, the team prioritized system-level integration testing and usability validation over comprehensive unit test coverage, given time constraints and

the need to validate the AI agent's effectiveness in the broader system context. Unit testing infrastructure remains in place for future development iterations.

- **Continuous Integration (CI):** GitHub Actions has been implemented to run automated tests, linters, and build checks upon every push or pull request.
- **Static Analysis:** SonarQube will be used to analyze the codebase, identify code smells, potential bugs, and security vulnerabilities.
- **Code Coverage:** Tools like Coverage.py (for Python) will be integrated into the CI pipeline to report and track unit test coverage metrics, aiming for a minimum of 80% coverage for core logic modules.
- **Linters:** `flake8` will enforce the PEP 8 standard for all Python backend code. ESLint will enforce the Google Style Guide for the JavaScript/React frontend.

3.7 Software Validation

Software validation ensures the final product meets the actual needs of the end-users (players) and stakeholders.

- **Revision 0 Demonstration with Supervisor:** The Revision 0 Demonstration (Feb. 2) will be specifically used to validate that the core functionality and *AI* performance meet the project goals. Dr. Istvan David will provide critical feedback on the *AI*'s strategic viability and adherence to the project scope. A key success metric for this validation is that all intended Revision 0 requirements are demonstrably met, with no major unresolved scope or functionality gaps identified.
- **User-Centric Validation (Usability Survey):** A partial usability survey will be conducted (using the final class presentation or an informal user testing session with competitive *Catan* players) to validate the *NFR* related to usability (NFR.S.2) and the clarity of the *AI* advice. Survey questions will be included in the usability testing extra. The target metric is an average rating of at least 4/5 for clarity, ease of navigation, and understanding of *AI* feedback.

- **External Data Validation:** The *RL* agent's performance will be validated longitudinally by running thousands of game simulations and comparing its win rate and strategic output against existing benchmark *Catan* opponents (e.g., baseline bots), and potentially against human gameplay data provided by competitive players. The primary metric for this validation is improved win rate and stable strategic output over the benchmark runs.

4 System Tests

This section details the system-level tests for validating the functional and non-functional requirements of the AlgoCatan system. These tests are designed to be run once the integrated system is complete, using representative data to simulate actual gameplay and training cycles.

4.1 Tests for Functional Requirements

The tests below are categorized by the major functional modules defined in Section S.2 of the SRS to ensure 100% coverage.

4.1.1 Computer Vision Model (S.2.1)

Status: NOT IMPLEMENTED (Out of Scope / Deferred)

1. T.FR.CV.1.1 (DEFERRED)
   Initial State: Physical board setup with known assets.
   Input: Captured camera feed.
   Output: Structured data matching physical state with 99.99% accuracy.
   Derivation: Verifies *FR.S*.1.1, *FR.S*.1.2, *FR.S*.1.3, *FR.S*.1.4, *FR.S*.1.5.

4.1.2 Training Pipeline & AI Model (S.2.2 & S.2.3)

1. T.FR.AI.2.1
   Control: Automatic
   Initial State: AI Model loaded into the Training Pipeline.
   Input: Trigger self-play training session using Elo League for opponent selection.

Output: System executes moves following official Catan rules ($FR.S.2.1$), generates success rewards ($FR.S.2.3$), and saves model snapshots.
Derivation: Verifies $FR.S.2.1$, $FR.S.2.2$, $FR.S.2.3$, $FR.S.2.4$, $FR.S.2.5$, $FR.S.3.5$.

2. T.FR.AI.2.2
Control: Automatic
Initial State: A GameState where only one legal move exists (e.g., specific road placement).
Input: Process state through the AI Model.
Output: AI returns the single valid action consistent with Catan rules.
Derivation: Verifies $FR.S.3.1$, $FR.S.3.2$.

3. T.FR.AI.2.3
Control: Automatic
Initial State: 100 matches played against a "Random Move" bot.
Output: The AI Model must achieve a win rate $> 50\%$.
Derivation: Verifies $FR.S.3.4$.

4. T.FR.AI.2.4
Control: Manual
Initial State: UI is active and a game is in progress using "Model v1.0".
Input: User uses the interface to select and load "Model v2.0" from the Elo League list mid-game.
Output: The system switches models without state loss; subsequent advice is generated by the newly selected model.
Derivation: Verifies $FR.S.3.3$.

4.1.3 Elo League System (S.2.4)

1. T.FR.ELO.3.1
Control: Automatic
Initial State: League contains 125 model snapshots. Model A (1200 Elo) plays Model B (1000 Elo).
Input: Model A wins the match.
Output: Elo ratings update via standard formula ($FR.S.3.7$). The oldest/weakest model is pruned to maintain the 125-cap ($FR.S.3.10$).
Derivation: Verifies $FR.S.3.6$, $FR.S.3.7$, $FR.S.3.8$, $FR.S.3.9$, $FR.S.3.10$.

4.1.4 Curriculum Learning Manager (S.2.5)

1. T.FR.CURR.4.1
   Control: Automatic
   Initial State: Training in Phase 1 (5 VP target). Advancement threshold set to 60% win rate.
   Input: Training metrics reach 61% win rate over 50 episodes.
   Output: System logs transition and automatically advances to Phase 2 (10 VP target).
   Derivation: Verifies *FR.S*.3.12, *FR.S*.3.13, *FR.S*.3.14, *FR.S*.3.15, *FR.S*.3.16.

4.1.5 User Interface (S.2.6)

1. T.FR.UI.5.1
   Control: Manual
   Initial State: Web interface open on both desktop and mobile devices.
   Input: User selects an opponent from the Elo League list and performs a build action.
   Output: UI displays the board in real-time (*FR.S*.4.1), reflects the build action immediately (*FR.S*.4.2), and scales correctly to the mobile screen size (*FR.S*.4.5).
   Derivation: Verifies *FR.S*.4.1, *FR.S*.4.2, *FR.S*.4.3, *FR.S*.4.5, *FR.S*.4.7, *FR.S*.4.8.

4.1.6 Game State Database & Manager (S.2.7 & S.2.9)

1. T.FR.DATA.6.1
   Control: Automatic
   Initial State: Active game session in progress.
   Input: A player performs a "Build City" action; the system process is then forcibly terminated (simulated crash).
   Output: Upon system restart, the Game State Manager successfully recovers the session from the database. The board state accurately reflects the built City and updated resource inventories (*FR.S*.5.4).
   Derivation: Verifies *FR.S*.5.1, *FR.S*.5.2, *FR.S*.5.4, *FR.S*.7.1, *FR.S*.7.2, *FR.S*.7.4, *FR.S*.7.5.

1. T.FR.DATA.6.2

Control: Automatic
Initial State: The Game State Database contains at least one record of a 50+ turn game.
Input: A request is sent via the API to retrieve the structured history (specifically dice rolls and turn transitions) for the session.
Output:
a) The database returns the complete, timestamped log of all dice rolls and turn transitions without data loss ($FR.S.7.3$).
b) The structured data is formatted for replay/analysis and returned within the efficiency targets for read operations ($FR.S.5.3$, $FR.S.5.5$).
Derivation: Verifies $FR.S.5.3$, $FR.S.5.5$, $FR.S.7.3$.

4.1.7 Backend API Server (S.2.8)

1. T.FR.API.7.1
   Control: Automatic
   Initial State: System under normal load.
   Input: UI sends a move request to the AI via the API.
   Output: UI update latency is $< 100ms$ and AI decision return is $< 1s$.
   Derivation: Verifies $FR.S.6.1$, $FR.S.6.2$, $FR.S.6.3$.

4.1.8 Explanation Pipeline (S.2.10)

1. T.FR.EXP.8.1
   Control: Manual
   Initial State: AI makes a move (e.g., placing a robber on a Wheat tile).
   Input: User clicks "Explain Move" in the UI.
   Output: The LLM generates a natural language response which is displayed in the UI.
   Derivation: Verifies $FR.S.4.4$, $FR.S.4.6$, $FR.S.8.1$, $FR.S.8.2$, $FR.S.8.3$.

4.2 Traceability Between Test Cases and Requirements

Table 2: Comprehensive Functional Requirement Traceability Matrix

Test Case ID	Requirements Verified
T.FR.CV.1.1	FR.S.1.1, FR.S.1.2, FR.S.1.3, FR.S.1.4, FR.S.1.5 (DEFERRED)
T.FR.AI.2.1	FR.S.2.1, FR.S.2.2, FR.S.2.3, FR.S.2.4, FR.S.2.5, FR.S.3.5
T.FR.AI.2.2	FR.S.3.1, FR.S.3.2
T.FR.AI.2.3	FR.S.3.4
T.FR.AI.2.4	FR.S.3.3
T.FR.ELO.3.1	FR.S.3.6, FR.S.3.7, FR.S.3.8, FR.S.3.9, FR.S.3.10
T.FR.CURR.4.1	FR.S.3.12, FR.S.3.13, FR.S.3.14, FR.S.3.15, FR.S.3.16
T.FR.UI.5.1	FR.S.4.1, FR.S.4.2, FR.S.4.3, FR.S.4.5, FR.S.4.7, FR.S.4.8
T.FR.DATA.6.1	FR.S.5.1, FR.S.5.2, FR.S.5.4, FR.S.7.1, FR.S.7.2, FR.S.7.4, FR.S.7.5
T.FR.DATA.6.2	FR.S.5.3, FR.S.5.5, FR.S.7.3
T.FR.API.7.1	FR.S.6.1, FR.S.6.2, FR.S.6.3
T.FR.EXP.8.1	FR.S.4.4, FR.S.4.6, FR.S.8.1, FR.S.8.2, FR.S.8.3

5 Tests for Nonfunctional Requirements

This section outlines the system-level tests for validating the nonfunctional requirements (NFRs) defined in Section S.2.8 of the SRS. These include scalability, usability, maintainability, installability, data integrity, and availability.

Each subsection corresponds to a major nonfunctional quality attribute of the AlgoCatan system. Some tests produce summary statistics or qualitative evaluations (e.g., graphs or surveys) rather than strict pass/fail outcomes.

Accuracy-related nonfunctional qualities are validated implicitly through the functional tests defined in Section 4.2 (T.FR.AI.2.1–2.3 and T.FR.CV.1.1–1.2), which record relative error and correctness metrics.

5.1 Performance and Scalability

This area ensures acceptable responsiveness and throughput under varying load.

Test Case: Baseline Performance

1. T.NFR.P.1
   Control: Automatic

   Initial State: AlgoCatan server running full stack with no concurrent load.

   Input/Condition: Single 4-player game with standard actions (build, trade, roll dice, 100+ repetitions).

   Performance Thresholds:

   - AI Move Suggestion: $\leq$ 500ms avg, $\leq$ 750ms peak
   - Board State Update: $\leq$ 200ms avg, $\leq$ 300ms peak
   - UI Render: $\leq$ 50ms/frame

   How tested: Automated with server-side logging and Chrome DevTools Performance API.

   Trace: NFR.S.1

Test Case: Scalability Under Load

1. T.NFR.P.2
   Control: Automatic

   Initial State: AlgoCatan server running full stack.

   Input/Condition: Simulate 1, 3, and 5 concurrent games. Measure response time scaling.

   Scalability Thresholds (Sub-Linear Growth):

   - Response time at 3 games $\leq 1.5\times$ single game
   - Response time at 5 games $\leq 2.0\times$ single game
   - CPU utilization $\leq 80\%$ at 5 games
   - No memory leaks or OOM errors over 1+ hour

How tested: Automated load-testing scripts with system monitoring. Pass if all thresholds met.

Trace: NFR.S.1

5.2 Usability

This area focuses on confirming that the interface is intuitive and accessible to users with minimal training. It also includes accessibility checks for users with visual impairments.

Test Case: Usability Survey and User Feedback

1. T.NFR.U.1
   Control: Manual

   Initial State: Final prototype deployed in a browser.

   Input/Condition: 5–10 participants familiar with *Catan* complete a observational study and fill out the usability survey (Appendix 7.2).

   Output/Result: Average rating $\geq 4/5$ for clarity, ease of navigation, and intuition.

   Test Case Derivation: Verifies NFR.S.2 (Usability) through direct user evaluation.

   How test will be performed: Conducted during the final demo; results aggregated and visualized as a bar chart.

Test Case: Accessibility and Visual Clarity

1. T.NFR.U.2
   Control: Manual

   Initial State: AlgoCatan UI loaded in browser.

   Input/Condition: Apply color-blindness filters and vary display scaling.

   Output/Result: All critical UI elements remain legible; contrast ratio $\geq 4.5 : 1$.

   Test Case Derivation: Verifies accessibility and clarity under varied visual conditions.

   How test will be performed: Tested using Chrome DevTools accessibility simulator; screenshots recorded for documentation.

5.3 Maintainability

This area ensures the modular design and structure of the codebase support efficient updates and debugging.

Test Case: Code Quality and Modular Design Inspection

1. T.NFR.M.1
   Control: Static

   Initial State: Complete source code committed to GitHub.

   Input/Condition: Run `SonarQube` and `flake8` to generate maintainability and style reports.

   Output/Result: No high-severity issues; modules follow consistent structure and clear separation of concerns.

   Test Case Derivation: Verifies NFR.S.3 (Maintainability) through code inspection and static analysis.

   How test will be performed: Peer inspection led by group members; issues logged in GitHub for resolution.

5.4 Installability

This area validates that AlgoCatan installs and operates correctly across all supported operating systems and browsers.

Test Case: Cross-Platform Installation Validation

1. T.NFR.I.1
   Control: Manual

   Initial State: Packaged release available.

   Input/Condition: Install and execute AlgoCatan on Windows 11 and macOS using Chrome and Firefox browsers.

   Output/Result: Application installs and runs successfully on all tested platforms.

   Test Case Derivation: Verifies NFR.S.4 (Installability) by ensuring cross-platform compatibility.

   How test will be performed: Each team member installs on a unique OS following a checklist; issues recorded and documented.

5.5 Data Integrity

This area ensures that the Game State Database maintains consistent, valid, and accurate data under all conditions, including normal operation, concurrent updates, and unexpected failures. This includes validation of incoming updates and transactional consistency to prevent corruption.

Test Case: Transaction Integrity and Recovery

1. T.NFR.D.1
   Control: Automatic

   Initial State: Active game session connected to the database.

   Input/Condition: Execute 10 consecutive state updates with an artificial network interruption during one update.

   Output/Result: Database remains consistent; interrupted transaction is rolled back cleanly; no other game states are affected.

   Test Case Derivation: Verifies transactional consistency portion of NFR.S.5 under failure conditions.

   How test will be performed: Automated integration test simulates network failures and verifies database consistency after each run.

Test Case: Data Validation on Updates

1. T.NFR.D.2
   Control: Automatic

   Initial State: Active game session connected to the database.

   Input/Condition: Attempt to submit invalid game state updates, such as negative resource counts, illegal moves, or missing required fields.

   Output/Result: Invalid updates are recognized and rejected.

   Test Case Derivation: Verifies validation portion of NFR.S.5, preventing data corruption from malformed inputs.

   How test will be performed: Automated tests inject invalid game state objects and verifies rejection.

Test Case: Concurrent Update Handling

1. T.NFR.D.3
   Control: Automatic

   Initial State: Two or more simultaneous game sessions active.

   Input/Condition: Simulate concurrent updates to overlapping resources (e.g., two players updating the same tile).

   Output/Result: Only one valid transaction is committed; conflicts are detected and resolved; no data corruption occurs.

   Test Case Derivation: Verifies database integrity under concurrent operations, completing NFR.S.5 coverage.

   How test will be performed: Automated integration tests run multiple simulated clients updating shared resources; system logs and database state are verified for correctness.

5.6 Availability

This area verifies that the system maintains acceptable availability during use, consistent with the SRS requirement that the system have an uptime of at least 99%.

Test Case: Service Availability Monitoring

1. T.NFR.AV.1
   Control: Automatic

   Initial State: System deployed and running under normal usage conditions.

   Input/Condition: Monitor backend service availability over the test period, including normal gameplay, replay access, and explanation requests.

   Output/Result: The backend API remains available for at least 99% of the observed test period.

   Test Case Derivation: Verifies NFR.S.6 (Availability).

   How test will be performed: Use uptime logs, health checks, and request success/failure counts during the evaluation period to compute observed service availability.

5.7 Accuracy and Correctness (Referenced)

Accuracy-related NFRs are verified by functional tests defined in Section 4.2:

- T.FR.AI.2.1–2.3 (*AI* Model and Recommendation Logic)
- T.FR.CV.1.1 (Computer Vision and Game State Capture)

These tests report relative error between predicted and expected outcomes. No separate nonfunctional test cases are required.

5.8 Traceability Between Test Cases and Requirements

This subsection is limited to non-functional requirement traceability; the functional requirement traceability matrix is presented earlier in the Functional Requirements tests section.

Table 3: Non-Functional Requirement Test Case Traceability Matrix

Test Case ID	**Requirement(s) Verified**
T.NFR.P.1	NFR.S.1 – Performance Baseline
T.NFR.P.2	NFR.S.1 – Scalability Under Load
T.NFR.U.1	NFR.S.2 – Usability Survey
T.NFR.U.2	NFR.S.2 – Accessibility
T.NFR.M.1	NFR.S.3 – Maintainability
T.NFR.I.1	NFR.S.4 – Installability
T.NFR.D.1	NFR.S.5 – Transaction Consistency
T.NFR.D.2	NFR.S.5 – Data Validation
T.NFR.D.3	NFR.S.5 – Concurrency Handling
T.NFR.AV.1	NFR.S.6 – Availability (Recovery & Uptime)

6 Unit Test Description

6.1 Overview and Integration with System Tests

This section documents the unit testing activities performed to verify individual components and modules of the RLCatan system at the code level. These unit tests complement the system-level tests described in Section 4

(Functional Requirements Evaluation) by providing detailed verification of API endpoints, utility functions, and integration points.

The unit tests are organized by functional module and include both dynamic testing (automated tests) and static analysis. These tests are to be executed as part of the continuous integration pipeline using Python's unittest/pytest framework for backend tests and integration with the API server for endpoint testing.

6.2 Unit Testing Scope and Strategy

6.2.1 Scope

The unit testing scope covers the following components:

- Backend API endpoints and request/response handling
- Game State Manager and game creation/initialization
- Path resolution utilities for model asset loading
- Game analysis module integration with Monte Carlo Tree Search (MCTS) engine
- Database constraint validation and state management

Out of scope for unit testing:

- External library verification (e.g., Catanatron, OpenAI Gym)
- React frontend component testing (addressed through usability testing in Section 3)
- LLM explanation pipeline (addressed through manual testing in Section 3)
- Computer Vision module (deferred and not implemented)

6.2.2 Testing Strategy

The unit testing strategy employs both white-box and black-box approaches:

- **Black-box testing:** API endpoints are tested by sending HTTP requests and validating response structures and status codes without knowledge of internal implementation.
- **White-box testing:** Path resolution and game state validation functions are tested with knowledge of implementation to verify edge cases and internal state transitions.
- **Mocking and Isolation:** External dependencies (e.g., GameAnalyzer, database) are mocked to isolate units under test and verify behavior in controlled conditions.

6.3 API and Game Management Module

This module tests the backend API endpoints and game creation/initialization logic, supporting the system-level tests T.FR.AI.2.1, T.FR.API.7.1, and T.FR.DATA.6.1.

1. `test_get_bots_endpoint`

   Type: Functional, Dynamic, Automatic

   Initial State: The API server is initialized and ready to receive requests.

   Input: HTTP GET request to `/api/bots` endpoint.

   Expected Output: 200 OK status code and JSON response containing list of available bot objects with at least 10 bots (including standard bots like `catanatron_ab_2`, `random`, `human`).

   Actual Output: -

   Result: -

   Requirement Traceability: Supports FR.S.3.8 (exposing league members to UI), FR.S.4.7 (loading bot list), FR.S.4.8 (selecting opponents)

   Module Tested: Backend API Server, Elo League System

2. `test_stress_test_endpoint`

   Type: Functional, Dynamic, Automatic

   Initial State: The API server is initialized.

   Input: HTTP GET request to `/api/stress-test` endpoint.

   Expected Output: 200 OK status code and JSON object representing game state with essential structures: "nodes" and "edges".

   Actual Output: -

   Result: -

   Requirement Traceability: Supports FR.S.5.1 (storing complete game records), FR.S.7.5 (structured board access)

   Module Tested: Backend API Server, Game State Manager

3. `test_player_factory_custom_bot`

   Type: Functional, Dynamic, Automatic

   Initial State: Internal functions mocked to simulate custom bot "my_custom_bot".

   Input: POST request to `/api/games` with "BOT:my_custom_bot" in player list.

   Expected Output: 200 OK response containing valid `game_id`.

   Actual Output: -

   Result: -

   Requirement Traceability: Supports FR.S.2.5 (curriculum-guided RL with dynamic opponent selection), FR.S.3.3 (model versioning and interchange)

   Module Tested: Backend API Server, Training Pipeline, AI Model integration

4. `test_create_game_all_player_types`

   Type: Functional, Dynamic, Automatic

   Initial State: The API server is initialized.

   Input: Series of POST requests to `/api/games` with different player types (CATANATRON, VALUE_FUNCTION, HUMAN, PPO models).

Expected Output: 200 OK status code and valid `game_id` for each request type.

Actual Output: -

Result: -

Requirement Traceability: Supports FR.S.2.1 (rule enforcement in training), FR.S.2.2 (self-play with league opponents), FR.S.3.1 (valid game state analysis)

Module Tested: Backend API Server, Game State Manager, Training Pipeline

6.4 Path Resolution Module

This module tests utility functions for safely resolving model asset paths across different deployment environments (HPC, local development, production).

1. `test_resolve_model_path_absolute_hpc`

   Type: Functional, Dynamic, Automatic

   Initial State: Path resolution utilities loaded.

   Input: Absolute file path string typical of HPC environment: `/nfs/features/rlcatan/models/v1.0`.

   Expected Output: Normalized path rebased to local application directory structure: `/app/models/v1.0`.

   Actual Output: -

   Result: -

   Requirement Traceability: Supports FR.S.2.2 (self-play training with correct model loading), FR.S.3.3 (seamless model interface)

   Module Tested: AI Model, Training Pipeline (path resolution utility)

2. `test_resolve_model_path_relative`

   Type: Functional, Dynamic, Automatic

   Initial State: Path resolution utilities loaded.

   Input: Standard relative path string: `data/models/v2`.

Expected Output: Absolute path correctly appending input to application root: `/app/data/models/v2`.

Actual Output: -

Result: -

Requirement Traceability: Supports FR.S.2.2 (opponent selection and model loading in training), FR.S.3.3 (model versioning)

Module Tested: AI Model, Elo League System (model lookup and loading)

6.5 Game Analysis Module

This module verifies integration with the Monte Carlo Tree Search (MCTS) engine for analyzing active games.

1. `test_mcts_analysis_endpoint`

   Type: Functional, Dynamic, Automatic

   Initial State: Valid game created via API; GameAnalyzer mocked to return fixed win probabilities (e.g., RED=0.5, BLUE=0.3, WHITE=0.15, ORANGE=0.05).

   Input: GET request to game's analysis URL: `/api/games/game_id/mcts-analysis`.

   Expected Output: 200 OK status and JSON response with `success=True` and win probabilities matching mocked values.

   Actual Output: -

   Result: -

   Requirement Traceability: Supports FR.S.3.4 (demonstrating AI learning through win probability analysis), FR.S.2.4 (batch and real-time evaluation modes)

   Module Tested: Backend API Server, Game State Manager, Game Analysis integration

2. `test_mcts_analysis_endpoint_not_found`

   Type: Functional, Dynamic, Automatic

   Initial State: The API server is initialized.

Input: GET request to analysis endpoint with non-existent game ID: `/api/games/99999/mcts-analysis`.

Expected Output: Error status code (404 Not Found or 500 Internal Server Error) without returning false success.

Actual Output: -

Result: -

Requirement Traceability: Supports robust error handling for FR.S.7.5 (structured game state access)

Module Tested: Backend API Server (error handling)

6.6 Database Constraint Validation Tests

The following tests were executed as part of system-level test T.NFR.D-3 (Data Integrity - Concurrent Update Handling) and verify that the database layer enforces game state and action validity at the unit level.

1. `test_invalid_game_state_rejection`

   Type: Functional, Dynamic, Automatic

   Initial State: Database with transaction logging enabled.

   Input/Condition: Attempt to store 15 intentionally invalid game states:

   - Negative resource counts
   - Invalid player counts (0 or ¿4)
   - Out-of-bounds settlement locations
   - Impossible board configurations

   Expected Output: 100% rejection of invalid states; no invalid data persists in database.

   Actual Output: -

   Result: -

   Requirement Traceability: Supports NFR.S.5 (Data Integrity - validation), FR.S.7.2 (accurate asset tracking)

   Module Tested: Game State Database, Game State Manager

2. `test_invalid_action_rejection`

   Type: Functional, Dynamic, Automatic

   Initial State: Active game session with action logging.

   Input/Condition: Attempt to store 12 intentionally invalid actions:

   - Malformed move objects (missing required fields)
   - References to non-existent game IDs
   - Illegal moves (building where not allowed)
   - Out-of-range action types

   Expected Output: 100% rejection of invalid actions; constraint violations logged.

   Actual Output: -

   Result: -

   Requirement Traceability: Supports FR.S.7.1 (rule-consistent state updates), FR.S.7.4 (automatic board state reflection)

   Module Tested: Game State Database, Backend API Server (input validation)

References

Rebecca DiFilippo, Jake Read, Matthew Cheung, and Sunny Yao. Hazard analysis. https://github.com/SY3141/RLCatan, 2025a.

Rebecca DiFilippo, Jake Read, Matthew Cheung, and Sunny Yao. System requirements specification. https://github.com/SY3141/RLCatan, 2025b.

7 Appendix

7.1 Symbolic Parameters

The definition of the test cases calls for the following SYMBOLIC_CONSTANTS. Defining these here ensures that performance thresholds and iteration counts are applied consistently across the system-level test suite.

Table 4: Symbolic Constants for System Tests

Constant	Value	Description (Associated Test Cases)
$T_{\text{AI\_LAT}}$	1 second	Maximum response time for AI move suggestions (Used in T.FR.API.7.1).
$T_{\text{UI\_LAT}}$	100 ms	Maximum latency for real-time UI synchronization (Used in T.FR.API.7.1).
N_{LEAGUE}	125	Capacity limit for the Elo League model registry (Used in T.FR.ELO.3.1).
N_{BATCH}	100	Number of Game State samples required for model benchmarking (Used in T.FR.AI.2.1).
N_{BOARD}	5	Minimum unique board configurations for setup verification (Used in T.FR.CV.1.1).
$N_{\text{SIM\_RUNS}}$	10	Number of automated iterations required to verify move logic and rule compliance (Used in T.FR.AI.2.1).

7.2 Usability Survey Questions

This section provides the planned questions for the usability survey, which will be used to validate the usability Non-Functional Requirement ($NFR.S.2$) and gather feedback on the clarity of the real-time board visualization ($FR.S.4.1$), the intuitiveness of player interactions ($FR.S.4.3$), and the understandability of AI-generated move explanations ($FR.S.8.3$), primarily targeting the Novice and Intermediate player profiles.

*NOTE: **Please see more about this in our extra** under /docs/Extras/UsabilityTesting as we did two studies with 7 participants in total, the first round we didnt have the LLM explanations, and after feedback from the observational think aloud study and the following survery below,

we added them. We will be comparing the results of the two studies to see if there is a significant difference in the usability ratings with and without the LLM explanations, which will help us understand the impact of this feature on experiential learning FR.S.8.3.

1. **Overall Experience:** "Overall, how was your experience with the interface? Please share any general recomendations or feedback you have about the usability and design of the interface."
2. **Clarity:** "On a scale of 1 to 5, how clear was the interface with what is happening in the game, allowing you to easily understand the game state and the actions being taken?"
3. **Game Flow:** "On a scale of 1 to 5, how well did the interface support the flow of the game, allowing you to easily follow and take actions without feeling lost or confused?"
4. **Intuition:** "On a scale of 1 to 5, how intuitive was the interface in matching the mechanics of the board game, making it easy for you to learn and play without needing extensive instructions?"

Appendix — Reflection

The information in this section will be used to evaluate the team members on the graduate attribute of Lifelong Learning.

The purpose of reflection questions is to give you a chance to assess your own learning and that of your group as a whole, and to find ways to improve in the future. Reflection is an important part of the learning process. Reflection is also an essential component of a successful software development process.

Reflections are most interesting and useful when they're honest, even if the stories they tell are imperfect. You will be marked based on your depth of thought and analysis, and not based on the content of the reflections themselves. Thus, for full marks we encourage you to answer openly and honestly and to avoid simply writing "what you think the evaluator wants to hear."

Please answer the following questions. Some questions can be answered on the team level, but where appropriate, each team member should write their own response:

1. What went well while writing this deliverable?

2. What pain points did you experience during this deliverable, and how did you resolve them?

3. What knowledge and skills will the team collectively need to acquire to successfully complete the verification and validation of your project? Examples of possible knowledge and skills include dynamic testing knowledge, static testing knowledge, specific tool usage, Valgrind etc. You should look to identify at least one item for each team member.

4. For each of the knowledge areas and skills identified in the previous question, what are at least two approaches to acquiring the knowledge or mastering the skill? Of the identified approaches, which will each team member pursue, and why did they make this choice?

Jake Read

1. This deliverable went fairly smoothly, mostly because we already had a clear set of requirements from the SRS to base the tests on. Unlike the previous deliverables, we split into two subteams. Matthew and

Rebecca worked primarily on this doc, while Sunny and I focused on getting started with the PoC, and then checked in with the VnV team to help review. I feel like this made the process more efficient, and less people working directly on one document meant fewer consistency issues.

2. Personally, I didn't have any major pain points with the VnV plan, but as I said before, I was mainly focused on the PoC and simply reviewed and helped organize VnV progress. I think Rebecca and Matthew did a great job, so the review process was quite smooth. It's possible they had some pain points I'm not aware of, but from my discussions with them this doc was more straightforward than the SRS.

3. For VnV, I think the team will need to acquire more knowledge about automated testing frameworks, particularly for Python and JavaScript. I'm not sure how experienced the others are in these areas, but when it comes to testing almost all my experience is in Java, so I think this will be a learning opportunity for me. We'll also need to familiarize ourselves with static analysis tools like SonarQube, as well as CI/CD pipelines using GitHub Actions. I can focus on SonarQube, since I'm interested in using it for future projects.

4. For learning about automated testing frameworks, we could either take online courses/tutorials or read official documentation and implement small projects to practice. I also previously used SonarQube briefly back in second year, so I could revisit that experience and supplement it with online resources. Of these options, I'll start with some online tutorials, and then reference what I did with some of my old work.

Rebecca DiFilippo

1. Writing this deliverable went fairly smoothly, mainly because we had a solid set of requirements from the SRS to guide the tests. Splitting into two subteams worked really well this time. Matthew and I focused on drafting the VnV document, while Sunny and Jake worked on the PoC and then helped review our work. I think this setup made things more organized and helped avoid overlapping edits and consistency issues. It also allowed us to make progress on the POC plan.

2. I didn't run into major pain points, though coordinating between the two subteams sometimes required a bit of back-and-forth to make sure everyone was aligned. Overall, Matthew and I were able to keep the doc consistent, and Jake and Sunny's feedback was really helpful for catching small issues early. The process felt much smoother than working on the SRS.

3. For VnV, the team will need to build more experience with automated testing frameworks.We also we'll need to get comfortable with static analysis tools like SonarQube and CI/CD pipelines through GitHub Actions. I'll focus on automated testing in Python since that's an area I want to strengthen for this project and future work which includes flake8 style checks.

4. To build these skills, we can look for online tutorials and courses, as well as consult with other people who have experience in these areas. I'll start with online tutorials for Python testing frameworks and flake8, as well as look for example projects that implement these tools effectively.

Matthew Cheung

1. Since the requirements were already written in the SRS, the process writing this document was fairly straight-forward. This felt far less stressful and complicated than the SRS since we now have a solid grasp on what project we are creating, and what the limitations and goals are for our project. As we continue to add more documentation, I feel this process will become even more frictionless as the project becomes even more well developed and thought out.

2. This document felt rather painless compared to the previous SRS. All of the techniques used to battle with LaTeX to build this document, were already encountered in the design of the previous SRS. This document felt more like an extension of the SRS, which meant that many of the issues that I faced here, I already encountered before. As we create more of these documents, our mastery increases, and with that comes the experience of dealing with pain points, so I think going forward, I expect to see even less friction when making the next document.

3. I will acquire and lead these human-centric methods by developing and piloting custom checklists for the project's interfaces. This approach is superior to just studying theory because it provides demonstrable and verifiable value immediately. By applying these checklists to the design document's data contracts (e.g., between the CV module and GameState Manager), I will ensure correctness and consistency before any implementation begins, reducing the cost of finding defects later.

Sunny Yao

1. Writing this deliverable went smoothly overall. Having a clear and detailed set of requirements from the SRS made it easier to structure and define the tests. The decision to split into two subteams, one focusing on the VnV doc and the other on the Proof of concept, helped improve efficiency and reduced overlapping edits or consistency issues. Compared to earlier deliverables like the SRS, this process felt more organized, and more straightforward, as the team now had a solid understanding of the project's goals and requirements.

2. There were few major pain points during this deliverable. The main challenge was coordinatin to ensure everyone stayed aligned with the rubric and the document remained consistent. However, frequent check-ins and reviews helped address issues early. Since most members were already familiar with LaTeX and the documentation process from the SRS, this deliverable felt smoother and more manageable. Overall, any difficulties were minor and quickly resolved through feedback meetings.

3. The team needs to deepen its understanding of automated testing frameworks, especially for Python and JavaScript, since testing will be a central focus of the VnV phase. Additionally, learning to use static analysis tools like SonarQube and setting up CI/CD pipelines through GitHub Actions will be important for maintaining code quality and automating verification. We plan to learn to use code analysis tools such as SonarQube, automated testing frameworks, and style checking tools like linters, to strengthen the group's skill set and development experience.

4. I plan to build these skills through a combination of online tutorials and official documentation. Some members can also revisit prior ex-

perience with tools like SonarQube, while others will explore Python testing frameworks and CI/CD setup through GitHub Actions. The group also intends to share findings and resources internally, leveraging peer learning to accelerate progress. This practical, example-driven approach will help ensure that the team can confidently apply these tools to the project.